WORKING DRAFT · UPANZI · 24 SEPTEMBER 2026

More code, slower tangle

AI adoption across twenty MOSIP core services: what changed in code volume, code quality and code-review practice, reported in absolute terms.

20 repositories 326 repo-months 6,786 commits 5,603 pull requests window 2025-01 → 2026-08

---

1 · INTRODUCTION

Motivation

The claim that AI coding assistants trade short-term speed for long-term maintainability is now widely repeated and has recent empirical support. He et al. [1] report that Cursor adoption raises velocity while worsening complexity growth across a large population of open-source projects, estimating an *average* effect using difference-in-differences against matched controls.

This work asks a narrower question with a different unit of analysis: what happens inside *one* organisation? MOSIP is digital public infrastructure — the identity platform deployed by several national governments — so its code is long-lived, security-sensitive, and maintained by a stable engineering organisation rather than a churning volunteer pool. If AI assistance degrades maintainability anywhere, this is a setting where the consequences matter.

MOSIP also proves to be an unusually clean natural setting, for a reason that emerged only during data collection: the organisation adopted **two different classes of AI tool, seven months apart**. An AI code *reviewer* arrived first, organisation-wide; AI code *writers* followed. These are distinct interventions with distinct predicted effects, and prior work generally collapses them.

> **RQ1 — Volume.** Did the amount of code, and the rate at which it is produced, change after AI coding agents were adopted?
>
> **RQ2 — Quality.** Did internal code quality — complexity, code smells, technical debt — change after adoption?
>
> **RQ3 — Review practice.** Did an AI code-review bot substitute for human review effort, or add to it?

2 · METHODS

Subjects, data and design

2.1 Repository selection

The MOSIP GitHub organisation holds 109 public repositories, 97 of them neither archived nor forks. Ranking by stars is uninformative here — the whole organisation spans 4 to 87 stars — and promotes artefacts with no analysable code, such as `documentation` (794 MB of prose) and the empty `mosip` meta-repository.

We instead ranked by **JVM source volume** obtained from the GitHub languages API, after excluding by name mocks, test harnesses, deployment and infrastructure repositories, configuration repositories and documentation. That left 36 candidate core services, of which we took the **top 20 by source volume**, totalling 47.2 MB of JVM source. Checkout size was rejected as a criterion because it is inflated by history and binaries: `packet-manager` occupies 188 MB on disk but holds only 0.51 MB of Java.

2.2 Data collection

Three independent sources were collected in September 2026 over a window of 2025-01 to 2026-08 (Table 1).

Table 1: Data sources and coverage.

Source	Extracted	Volume
Git history	commits, lines ±, contributors	`6,786` commits
		`3.19M` lines added
GitHub API	pull requests, reviews, issues	`5,603` PRs
		`17,090` reviews
		`1,222` issues
SonarQube 26.8	ncloc, cognitive complexity, code smells, technical debt, bugs, vulnerabilities, duplication	`286` snapshots
		`285/326` (87%)

SonarQube reports only on a checked-out working tree, so history was reconstructed by checking out the last commit of each month and re-analysing the project in full — 286 independent analyses on a dedicated Linux host. Months in which nothing landed on the integration branch have no snapshot, accounting for nearly all of the 13% shortfall. Analysis ran on source without compilation, matching the procedure of [1].

Velocity is counted across *all* branches (`git log --all`), because MOSIP follows gitflow and `master` held only 2,025 of 6,786 commits in the window; quality snapshots necessarily follow a single integration branch. The two halves therefore describe different commit sets — "all work performed" versus "quality of the shippable tree" — and are not strictly commensurable.

2.3 Detecting adoption

Adoption was not self-reported. Three independent signal families were matched against every repository: commit-message trailers and identities (`Co-Authored-By: Claude`, `cursoragent`), agent configuration files committed to the tree (`CLAUDE.md`, `.cursor/`, `AGENTS.md`), and bot accounts acting on GitHub. Detection yielded **4,288 AI-attributable actions** (Table 2).

Table 2: Detected AI tools and organisation-wide adoption.

Tool	Class	Actions	Adoption
CodeRabbit	reviews code	4,141	**2025-11** (14/19 repos)
Cursor	writes code	64	2026-03 – 2026-08
OpenAI Codex	writes code	48	2026-04 – 2026-06
Claude Code	writes code	35	2026-06 – 2026-08

The gap between a repository's review-bot and coding-agent adoption has median **7 months** (range 4–9). RQ1 and RQ2 are therefore evaluated against **coding-agent** adoption and RQ3 against **review-bot** adoption. Collapsing the two attributes code-generation effects to the arrival of a commenting tool.

2.4 Analysis design

Each repository's timeline is re-indexed to event time,

$$\tau_{it} = t - a_i, \tag{1}$$

where a_i is repository i's own adoption month, and the twenty series are pooled by τ.

All measures are reported raw. No normalisation is applied: no ratio to a pre-adoption baseline, and no division by code size. Figures use logarithmic axes where repositories span orders of magnitude (`ncloc` ranges from 7,931 to 355,811), but that is a display choice and leaves the values untouched.

Every test is a *within-repository* comparison, so each repository serves as its own control and differences in scale between repositories do not confound the tests themselves. Scale does dominate the pooled box plots, where the height of each box is largely between-repository size rather than change over time; the per-repository view in Figure 2 is provided for that reason.

Hypothesis tests are computed at the **repository level**: each repository contributes one pre-adoption median and one post-adoption median, giving 16–19 independent pairs compared by Wilcoxon signed-rank. Pooling repository-months would inflate n to roughly 300 and overstate significance, since months within a repository are strongly autocorrelated. Pre is $\tau \in [-6, -1]$; post is $\tau \in [+1, +3]$ for RQ1–2 and $[+1, +6]$ for RQ3. The incomplete current month is excluded throughout.

3 · RESULTS

Results

3.1 Code volume

Table 3: Code volume, raw. Repository-level paired medians, Wilcoxon signed-rank. Tinted rows reached significance.

Measure (raw)	pre	post	ratio	p	n	up
Code size (ncloc)	45,362	50,531	1.11	**<.0001**	16	**16/16**
Lines removed / mo	403	501	1.24	**.007**	17	**14/17**
Lines added / mo	1,583	3,098	1.96	.145	17	10/17
Lines added / commit	243.8	435.7	1.79	.244	17	9/17
Commits / month	11.0	12.0	1.09	.287	17	11/17
Contributors / month	4.5	3.5	0.78	.726	17	5/17

FINDING 1

Code volume grew unanimously and significantly: every one of the 16 repositories with quality data was larger after adoption, a median increase of 11% within three months. Deletion activity also rose ($+24\%$, $p = .007$) in 14 of 17 repositories.

Commit *count* did not change ($p = .287$). Commit *size* nearly doubled in the median (244 $\rightarrow$ 436 lines added per commit) but is not significant and splits 9/17 — the median moved because a minority of repositories changed a great deal, not because most did. That measure is also a monthly mean (lines $\div$ commits) rather than a distribution over commits, so a single bulk import dominates its month; `nfiq` 2025-09 comprises 2 commits and 36,810 lines. It should be read as indicative only.

Contributors fell 4.5 $\rightarrow$ 3.5, far from significance, and in the same direction as an organisation-wide slowdown discussed in Section 4.

3.2 Code quality

Table 4: Code quality, raw totals.

Measure (raw)	pre	post	ratio	p	n	up
Cognitive complexity	2,703	2,762	1.02	**.005**	16	**14/16**
Code smells	790.5	827.5	1.05	**.011**	16	**13/16**
Duplication (%)	9.00	8.70	0.97	**.024**	16	3/16
Vulnerabilities	8.5	9.5	1.12	.093	16	5/16
Bugs	14.0	14.0	1.00	.063	16	6/16
Technical debt (min)	5,554	5,617	1.01	.496	16	9/16

FINDING 2

Absolute complexity and code smells rose significantly: complexity in 14 of 16 repositories ($+2\%, p = .005$) and smells in 13 of 16 ($+5\%, p = .011$). Technical debt, bugs and vulnerabilities did not move. Duplication *fell* ($-3\%, p = .024$) in 13 of 16 repositories.

The increases are consistent in direction but small in magnitude. Read alongside Finding 1, the same repositories over the same interval added **11% more code** while complexity rose **2%** and smells **5%**: the measured quality burden grew, and grew more slowly than the codebase producing it.

What absolute counts can and cannot settle

A reviewer will ask whether these increases reflect degrading code or simply more of it. Absolute counts alone cannot separate the two: any codebase that grows accumulates more complexity points and more smells even when every new line is written to the same standard as the old. We report absolute values throughout, so the question is raised here rather than answered.

The relevant evidence available is the juxtaposition above — 11% more code against a 2% rise in complexity. A size-conditioned analysis, or a model treating `ncloc` as a covariate, would settle it directly and is the obvious next step.

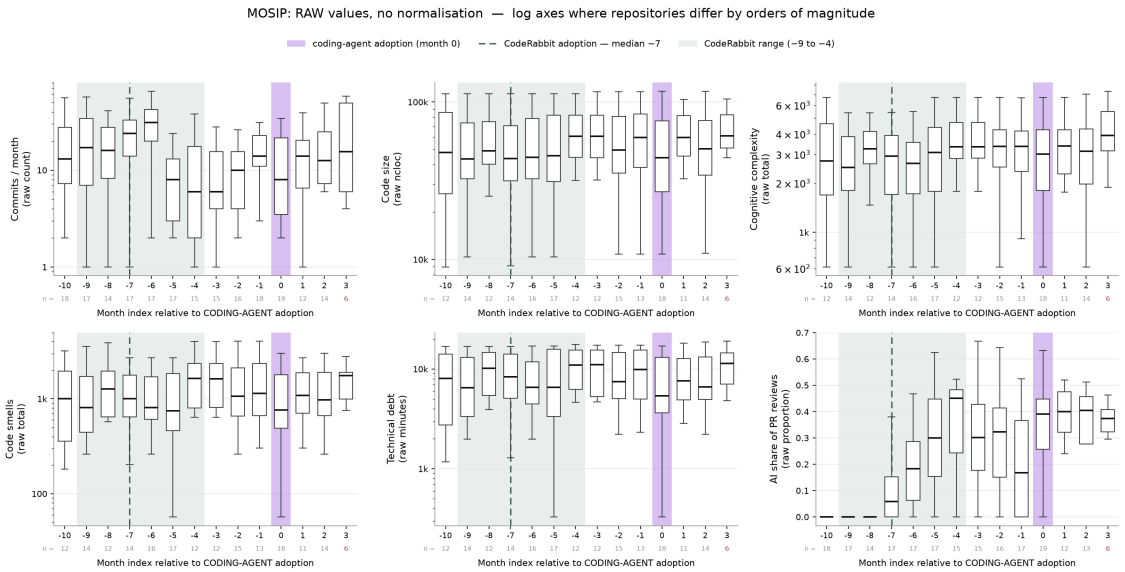


Figure 1: Event study on coding-agent adoption (purple band), all values raw. The earlier review-bot rollout appears as the dashed line at its median offset $\tau = -7$, with a band over its range. Axes are logarithmic where repositories differ by orders of magnitude. Box *height* here is dominated by between-repository size rather than change over time; n beneath each box, red below 10.

Per repository, 17 of 19 grew in size (Figure 2). Both directions of the complexity relationship are present and should not be averaged away.

3.3 Code-review practices

RQ3 is evaluated against **review-bot** adoption, the relevant intervention, with pre $= \tau \in [-6, -1]$ and post $= \tau \in [+1, +6]$. All counts are raw.

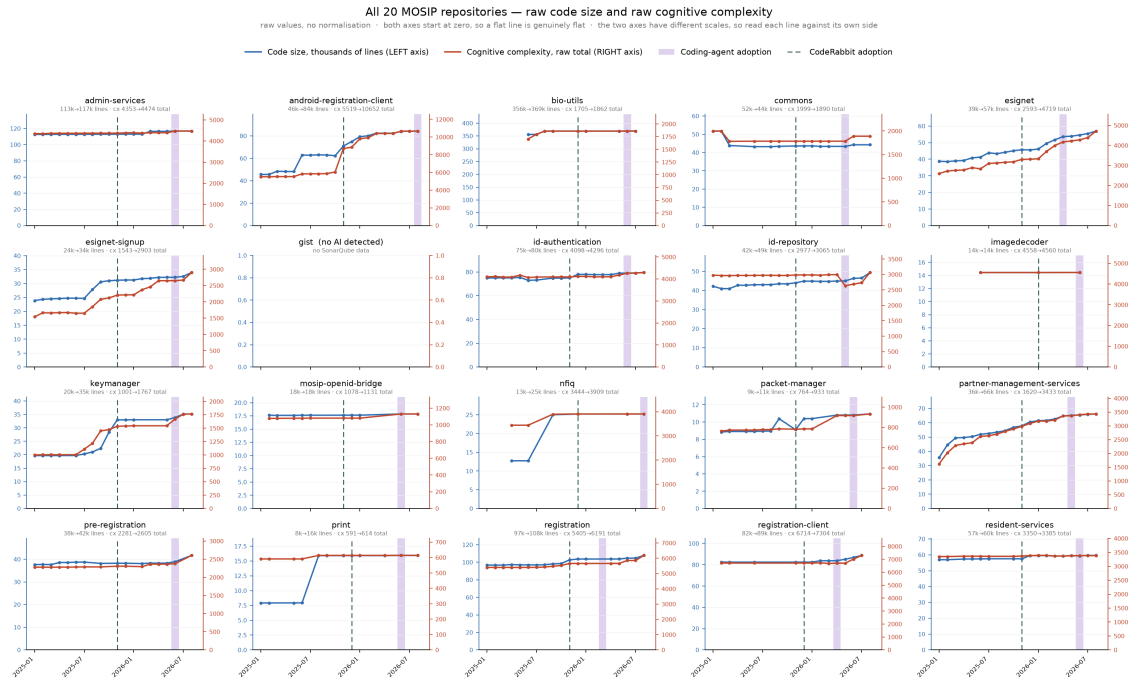


Figure 2: Per-repository view, raw values, both axes anchored at zero. In most repositories the two lines track each other, the signature of complexity accumulating in step with code. The exceptions run both ways: `print` doubled in size (8k → 16k lines) with complexity almost flat (591 → 614), while `esignet-signup` grew 42% in size (24k → 34k) but 88% in complexity (1,543 → 2,903).

Table 5: Review practice, split at review-bot adoption.

Measure (raw)	pre	post	ratio	p	n
AI reviews / month	0.0	6.0	–	**.001**	19
Human reviews / month	12.0	16.0	1.33	.260	19
All reviews / month	12.0	26.5	2.21	**.018**	19
Human reviews / PR	**1.43**	**1.80**	1.26	**.009**	19
PRs opened / month	12.5	8.0	0.64	**.006**	19

FINDING 3

The review bot was **additive, not substitutive**. Human review per pull request *rose* from 1.43 to 1.80 ($p = .009$), falling in only 6 of 19 repositories. Total review activity more than doubled. AI review accounts for **26.1%** of all 15,824 review submissions across the organisation.

This is the clearest result in the study and the least threatened by the design limitation in Section 4, because the treatment indicator and the outcome are the same class of event. The pre-adoption AI review share is exactly zero in every repository in every month and rises to a plateau of 0.35–0.45 after adoption, which also validates the detection procedure end to end.

One competing explanation

Pull-request volume fell 36% over the same period ($12.5 \rightarrow 8.0, p = .006$). Human

reviews per PR could therefore rise because fewer, larger pull requests each attract more attention rather than because reviewers became more engaged. Distinguishing these requires conditioning on PR size, which we have not yet done.

What this design cannot establish

1. **Adoption is near-simultaneous, so event time is nearly collinear with calendar time.** Fourteen of nineteen repositories adopted CodeRabbit in a single month; $\mathrm{corr}(\tau, t) = 0.997$ for the review wave and 0.980 for the coding wave. In the canonical two-way specification

$$y_{it} = \alpha_i + \lambda_t + \sum_{k \neq -1} \beta_k \mathbf{1}[\tau_{it} = k] + \varepsilon_{it}, \tag{2}$$

   common adoption $a_i = a$ implies $\mathbf{1}[\tau_{it} = k] = \mathbf{1}[t = a + k]$, a deterministic function of calendar time lying in the span of the time fixed effects, so β_k is not separately identified.

2. **Absolute counts confound quality with quantity.** Complexity and smell totals necessarily rise in a growing codebase. Findings 1 and 2 report real, separately measured quantities, but this study does not adjudicate whether the quality increases exceed what growth alone implies.

3. **An organisation-wide slowdown.** Mean output fell from 24.4 to 14.5 commits per repository per month between 2025 and 2026, beginning months before any coding agent. Every repository's pre/post boundary straddles it. We therefore advance no velocity claim.

4. **Short post-adoption window.** Coding agents arrived mid-2026 and the data ends 2026-08. By $\tau = +3$ only six repositories remain. Long-run maintainability effects — precisely what the literature predicts [1] — are not yet observable.

5. **Small treatment dose.** 147 commits are attributable to coding agents across 326 repository-months.

6. **Unbalanced panel.** Repositories enter and leave columns; only five appear at both $\tau = -10$ and $\tau = +3$. Because all tests are within-repository this does not bias them, but raw cross-column comparisons in the figures are unsafe.

7. **Single organisation.** MOSIP's engineering culture, review norms and gitflow process are its own. Nothing here generalises without replication.

What would establish causality

A matched cohort of comparable non-adopting repositories observed over the same calendar window. This is the only remedy for threat 1: because adoption is near-simultaneous, additional MOSIP repositories would share the same timing and the same shock, and would not help. The study's existing `matching_simple.py` and

> `analyze_repos_control.py` construct such a cohort, and `panel_ai_impact.csv` is already schema-compatible with the difference-in-differences and panel-GMM notebooks.

5 · SUMMARY

Where this leaves the hypothesis

Within one organisation, over a short post-adoption window, with a small treatment dose: the code grew by 11%, and its measured complexity and smell counts grew by 2% and 5%. Technical debt, bugs and vulnerabilities did not move; duplication improved. The direction is consistent with the population literature [1], but the magnitudes are substantially smaller than the growth in code volume over the same interval, and this design cannot separate either from a coincident organisational shock.

One contribution stands independently of the AI question. The distinction between AI reviewers and AI writers is empirically consequential: these tools arrived seven months apart, had different effects, and conflating them produces attribution errors. The review-practice result in Section 3.3 is also the most robust finding here, because the outcome and the intervention are the same class of event.

Next steps: construct a matched control cohort, which is the only remedy for the identification problem in Section 4; add a size-conditioned analysis so Finding 2 can be read against what growth alone predicts; recompute commit size as a true per-commit distribution, which also permits a within-repository comparison of AI-authored against human-authored commits and sidesteps the timing confound entirely; and re-run collection in six months, when the post-adoption window will support the long-run question.

Data availability

All figures and statistics are reproducible from the per-repository reports and the pooled panel `panel_ai_impact.csv` via the scripts in `_tools/analysis/`. Tests use `scipy.stats.wilcoxon`, two-sided, on repository-level pairs.

Draft — not for circulation. All values raw; no normalisation applied.

Descriptive before/after within a single organisation — not a causal estimate.

References

[1] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*, New York, NY, USA, 2026. ACM.